



# Deployment Guide

Step by step, from one GitHub repository to a live production site

---

Repository: [github.com/Ajay-Dabhi-99/A.AI](https://github.com/Ajay-Dabhi-99/A.AI)

Stack: Vercel (website) + Render (API) + Supabase (PostgreSQL, Storage) + Upstash (Redis) + Resend (email)

Version 1.0 · 17 September 2026 · Author: Dabhi Ajay

# Contents

|                                                             |    |
|-------------------------------------------------------------|----|
| 1. Read this first.....                                     | 3  |
| 1.1 What you will have at the end .....                     | 3  |
| 1.2 Why you must own a domain .....                         | 3  |
| 1.3 Accounts you need .....                                 | 3  |
| 1.4 A worksheet for your secret values .....                | 4  |
| 2. How one GitHub repository runs everything .....          | 5  |
| 2.1 Who reads what.....                                     | 5  |
| 2.2 The flow of a change .....                              | 5  |
| 2.3 Where every setting lives .....                         | 5  |
| 2.4 Branches.....                                           | 5  |
| 3. Step-by-step deployment .....                            | 7  |
| Phase A: prepare your computer and repository .....         | 7  |
| Phase B: domain .....                                       | 7  |
| Phase C: Supabase (database and storage).....               | 7  |
| Phase D: Upstash (Redis).....                               | 8  |
| Phase E: Resend (email).....                                | 9  |
| Phase F: AI provider keys .....                             | 9  |
| Phase G: the login secret .....                             | 9  |
| Phase H: Render (API server).....                           | 10 |
| Phase I: Vercel (website).....                              | 11 |
| Phase J: DNS check.....                                     | 12 |
| Phase K: GitHub settings .....                              | 12 |
| Phase L: first release .....                                | 13 |
| Phase M: after the first release.....                       | 14 |
| 4. Releasing updates later .....                            | 15 |
| 4.1 Database changes.....                                   | 15 |
| 4.2 Changing a secret.....                                  | 15 |
| 5. Optional: error monitoring with Sentry .....             | 16 |
| 6. Optional: adding a staging site from the same repo ..... | 17 |
| 7. Rollback, backups and restore.....                       | 18 |
| 7.1 Rolling back a bad release .....                        | 18 |
| 7.2 Backups .....                                           | 18 |
| 8. Troubleshooting.....                                     | 19 |
| Appendix A: all API environment variables .....             | 20 |
| Appendix B: useful commands.....                            | 21 |
| Appendix C: final checklist .....                           | 22 |

# 1. Read this first

This guide takes you from the code on your computer to a live A.ai website that real users can open. Every step says **where to click**, **what to type** and **how to check that it worked**. Follow the steps in order; later steps use values you copy in earlier ones.

Everything here is based on the files that are already in the repository: `render.yaml`, `apps/web/vercel.json`, `.github/workflows/ci.yml`, `.github/workflows/deploy.yml`, `.env.example` and `packages/config/src/server.ts`. You do not need to write any new code to deploy.

## 1.1 What you will have at the end

| Address                                 | What it is                                                     | Hosted on            |
|-----------------------------------------|----------------------------------------------------------------|----------------------|
| <code>https://app.yourdomain.com</code> | The A.ai website (React app)                                   | Vercel               |
| <code>https://api.yourdomain.com</code> | The A.ai API server (Fastify, Node 22)                         | Render (web service) |
| (no address)                            | Daily job that deletes expired uploaded files, 03:30 UTC       | Render (cron job)    |
| (private)                               | PostgreSQL database: accounts, chats, history, model registry  | Supabase             |
| (private)                               | File storage bucket <code>a-ai-attachments</code> for images   | Supabase Storage     |
| (private)                               | Redis: guest sessions, daily quotas, rate limits, job progress | Upstash              |
| (private)                               | Sign-up verification and password-reset emails                 | Resend               |

Throughout this guide, replace `yourdomain.com` with your real domain name.

## 1.2 Why you must own a domain

The API keeps users signed in with secure cookies (`__Host-` prefix, `Secure`, `HttpOnly`, `SameSite=Lax`). A browser sends those cookies to the API only when the website and the API share the same parent domain, for example `app.yourdomain.com` and `api.yourdomain.com`.

### IMPORTANT

Do not try to run production on the free addresses `something.vercel.app` and `something.onrender.com`. They are different sites, so the browser blocks the cookies: sign-in and guest chat will not work.

Buy a domain (any registrar: GoDaddy, Namecheap, Hostinger, Cloudflare, etc.) before you start.

## 1.3 Accounts you need

| Service          | Plan                                                          | Why                                                                                            |
|------------------|---------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| GitHub           | Free (repo is public)                                         | Holds the code; runs tests (CI) and the Deploy button                                          |
| Domain registrar | Any                                                           | The <code>app.</code> and <code>api.</code> addresses                                          |
| Supabase         | Free plan is enough to start (2 projects: production + test)  | Database and file storage                                                                      |
| Upstash          | Free plan is enough to start (2 databases: production + test) | Redis                                                                                          |
| Render           | <b>Paid Starter plan</b>                                      | API server. The migration step (pre-deploy command), cron jobs and Shell need a paid instance. |

| Service                                     | Plan                        | Why                                                                             |
|---------------------------------------------|-----------------------------|---------------------------------------------------------------------------------|
| Vercel                                      | Hobby (free)                | Website hosting                                                                 |
| Resend                                      | Free                        | Emails. Needs your domain verified.                                             |
| Google AI Studio (Gemini), Groq, OpenRouter | Free keys are fine to start | At least <b>one</b> is required. Groq also powers the microphone (voice input). |
| Sentry                                      | Optional, free              | Error reports                                                                   |

Free-plan limits and prices change; check each provider's pricing page before you sign up.

## 1.4 A worksheet for your secret values

As you go, you will copy about fifteen values. Keep them in a password manager or a private note, **never** in a file inside the repository and never in a chat or screenshot. Tick each one as you collect it:

| Value                                                                                     | You get it in | Goes to                                    |
|-------------------------------------------------------------------------------------------|---------------|--------------------------------------------|
| <code>DATABASE_URL</code> (production, port 6543)                                         | Step 7        | Render                                     |
| <code>DIRECT_URL</code> (production, port 5432)                                           | Step 7        | Render                                     |
| <code>SUPABASE_URL</code> , <code>SUPABASE_SERVICE_ROLE_KEY</code>                        | Step 9        | Render (API + cron)                        |
| <code>TEST_DATABASE_URL</code> , <code>TEST_DIRECT_URL</code>                             | Step 11       | GitHub environment <code>test</code>       |
| <code>REDIS_URL</code> (production)                                                       | Step 12       | Render                                     |
| <code>TEST_REDIS_URL</code>                                                               | Step 13       | GitHub environment <code>test</code>       |
| <code>RESEND_API_KEY</code> , <code>EMAIL_FROM</code>                                     | Step 15       | Render                                     |
| <code>GEMINI_API_KEY</code> / <code>GROQ_API_KEY</code> / <code>OPENROUTER_API_KEY</code> | Step 16       | Render                                     |
| <code>JWT_SECRET</code>                                                                   | Step 17       | Render                                     |
| Render deploy hook URL                                                                    | Step 22       | GitHub <code>RENDER_DEPLOY_HOOK_URL</code> |
| Vercel deploy hook URL                                                                    | Step 27       | GitHub <code>VERCEL_DEPLOY_HOOK_URL</code> |

## 2. How one GitHub repository runs everything

You have a single repository, [github.com/Ajay-Dabhi-99/A.AI](https://github.com/Ajay-Dabhi-99/A.AI). That is exactly what this project is designed for: it is a **monorepo**. The website, the API, the shared packages and the database migrations all live in the same repository, and every hosting service simply reads a different part of it.

### 2.1 Who reads what

| Service        | Connects to                         | Reads                                                                      | Builds / runs                                                                                           |
|----------------|-------------------------------------|----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| GitHub Actions | The repo itself                     | <code>.github/workflows/ci.yml</code> , <code>deploy.yml</code>            | Tests on every push/PR; the manual <b>Deploy</b> workflow                                               |
| Render         | Same repo, branch <code>main</code> | <code>render.yaml</code> (repo root)                                       | API ( <code>apps/api</code> ) and cleanup cron job; runs migrations from <code>prisma/migrations</code> |
| Vercel         | Same repo, branch <code>main</code> | Root Directory <code>apps/web</code> and <code>apps/web/vercel.json</code> | Website ( <code>apps/web</code> ), installing the whole workspace from the repo root                    |
| Supabase       | Nothing (no Git link)               | —                                                                          | Receives migrations from Render's pre-deploy step                                                       |

So you do **not** need separate repositories for frontend and backend. Render and Vercel both connect to the same repo; because `render.yaml` sets `autoDeploy: false` and Vercel automatic deployments will be switched off (Step 26), **nothing goes live just because you pushed code**. Releases happen only when you press the Deploy button in GitHub, after all tests pass.

### 2.2 The flow of a change

```

your computer --git push--> branch (e.g. develop or feature/x)
                        |
                        | CI: format, lint, types, tests, build, browser smoke
                        v
                        Pull Request --> merge into main (only when CI is green)
                        |
                        GitHub > Actions > Deploy > Run workflow > "production"
                        |
1. Full gate again (verify + browser smoke + live DB/Redis tests)
2. POST Render deploy hook -> Render builds API, runs migrations, swaps version
3. POST Vercel deploy hook -> Vercel builds and publishes the website
4. Waits until https://api.../health shows this commit, then runs the smoke test

```

### 2.3 Where every setting lives

| Kind of setting                                                                   | Stored in                                                    | Committed to Git? |
|-----------------------------------------------------------------------------------|--------------------------------------------------------------|-------------------|
| Build/start commands, region, non-secret env values                               | <code>render.yaml</code> , <code>apps/web/vercel.json</code> | Yes               |
| API secrets (database, Redis, keys, JWT)                                          | Render dashboard (Environment tab)                           | <b>Never</b>      |
| Website settings ( <code>VITE_API_URL</code> )                                    | Vercel dashboard (Environment Variables)                     | No                |
| Deploy hook URLs, test database URLs                                              | GitHub → Settings → Environments → Secrets                   | <b>Never</b>      |
| Public URLs for the smoke test ( <code>API_URL</code> , <code>WEB_ORIGIN</code> ) | GitHub → Environments → Variables                            | No                |
| Your local development values                                                     | <code>.env</code> in repo root (git-ignored)                 | <b>Never</b>      |

### 2.4 Branches

- `main`: the only branch that is ever deployed. Protect it (Step 32).

- `develop` or `feature/...`: where you work. CI runs on pushes to `main` and `develop` and on every pull request.
- Staging (a second, test copy of the live site) is **optional**. You can start with production only and add staging later from the same repo (Section 6).

## 3. Step-by-step deployment

### Phase A: prepare your computer and repository

#### Step 1 Install the tools

1. Install **Node.js 22** (22.13 or newer) from nodejs.org. Check: `node -v` shows **v22.x**.
2. Open a terminal and enable pnpm: `corepack enable`. Check: `pnpm -v` shows **10.x**.
3. Install **Git**. Check: `git --version`.

#### Step 2 Get the code and install dependencies

If the project is already on your computer (for example `H:\AI Arena`), open a terminal in that folder and skip the clone.

```
git clone https://github.com/Ajay-Dabhi-99/A.AI.git
cd A.AI
pnpm install
```

`pnpm install` also runs `prisma generate` automatically.

#### Step 3 Run the full check locally

```
pnpm verify
```

**Expected:** format, lint, typecheck, unit tests, integration tests and build all pass. Do not deploy while this fails.

#### Step 4 Push everything to GitHub and confirm CI is green

1. Commit your work and push: `git push origin main` (or push a branch and merge a pull request).
2. Open [github.com/Ajay-Dabhi-99/A.AI](https://github.com/Ajay-Dabhi-99/A.AI) → **Actions**. The **CI** workflow must show two green jobs: "Typecheck, lint, test, build" and "Browser smoke (Playwright, mocked API)".

#### NOTE

Render and Vercel build from what is on GitHub, not from your computer. Uncommitted changes are never deployed.

### Phase B: domain

#### Step 5 Prepare your domain

1. Buy the domain (for example `yourdomain.com`).
2. Log in to the registrar and find the **DNS / Manage DNS** page. Keep it open; you will add records in Steps 14, 21 and 28.
3. Decide the two addresses: `app.yourdomain.com` (website) and `api.yourdomain.com` (API).

### Phase C: Supabase (database and storage)

#### Step 6 Create the production project

1. Go to [supabase.com](https://supabase.com) → **Sign in** → **New project**.
2. Name: `a-ai-production`. Set a strong **database password** and save it in your worksheet.
3. Region: pick the one nearest your users. The API on Render uses **Singapore** by default (`render.yaml`), so choose **Southeast Asia (Singapore)** unless you change Render's region too.
4. Click **Create new project** and wait until it is ready.

### Step 7 Copy the two database connection strings

1. In the project, click **Connect** (top bar).
2. Copy the **Transaction pooler** string (port **6543**). Replace **[YOUR-PASSWORD]** with your database password. This is **DATABASE\_URL**.
3. Copy the **Direct connection** string (port **5432**), again with your password. This is **DIRECT\_URL**.
4. If your password has special characters such as **@ # / ?**, URL-encode them (for example **@** becomes **%40**), or choose a password with only letters and digits.

Both strings must start with **postgresql://** or **postgres://**. The API uses the pooled one; database migrations use the direct one (see **prisma.config.ts**).

### Step 8 Create the private storage bucket

1. Left menu → **Storage** → **New bucket**.
2. Name: **a-ai-attachments** (exactly this).
3. Leave **Public bucket OFF**. Click **Create**.

### Step 9 Copy the Supabase URL and service-role key

1. Left menu → **Project Settings** → **API** (or **API Keys**).
2. Copy **Project URL** (looks like **https://abcd1234.supabase.co**). This is **SUPABASE\_URL**.
3. Copy the **service\_role** secret key. This is **SUPABASE\_SERVICE\_ROLE\_KEY**.

#### IMPORTANT

The service-role key can read and write everything. It goes **only** into Render. Never put it in Vercel, in the website, or in Git.

### Step 10 Check backups

Left menu → **Database** → **Backups**. Confirm daily backups are listed. If your plan offers Point-in-Time Recovery, you can enable it here.

#### NOTE

You do not need to create tables by hand. The first Render deploy runs **pnpm db:deploy**, which applies all migrations in **prisma/migrations** and turns on Row Level Security for every table.

### Step 11 Create the test project (used only by automated tests)

1. Create a second project: **New project** → name **a-ai-test**, same region.
2. Copy its pooled string (port 6543) as **TEST\_DATABASE\_URL** and its direct string (port 5432) as **TEST\_DIRECT\_URL**.
3. No storage bucket is needed here.

#### IMPORTANT

Never use the production database for tests. The Deploy workflow runs **pnpm test:live**, which writes test data into this database. If these test values are missing, the Deploy workflow fails on purpose.

## Phase D: Upstash (Redis)

### Step 12 Create the production Redis database

1. Go to **console.upstash.com** → **Redis** → **Create Database**.



2. Name: **a-ai-production**. Region: the one nearest Render (for example **ap-southeast-1, Singapore**). Keep **TLS enabled**.
3. Open the database → **Connect** → choose **ioredis / TCP**. Copy the URL that starts with **rediss://**. This is **REDIS\_URL**.

**NOTE**

It must start with **rediss://** (two s = encrypted). With **redis://** the production API refuses to start.

**Step 13 Create the test Redis database**

Repeat Step 12 with the name **a-ai-test** and save the URL as **TEST\_REDIS\_URL**.

**Phase E: Resend (email)****Step 14 Verify your domain in Resend**

1. Go to **resend.com** → **Sign up** → **Domains** → **Add Domain**. Enter **yourdomain.com**.
2. Resend shows several DNS records (TXT for SPF, CNAME/TXT for DKIM, and an MX record). Add each one on your registrar's DNS page exactly as shown.
3. Back in Resend, click **Verify**. Wait until the status is **Verified** (can take from minutes up to a few hours).

**Step 15 Create the API key and choose the sender address**

1. **API Keys** → **Create API Key**, permission **Sending access**. Copy it: **RESEND\_API\_KEY**.
2. Choose the sender: **EMAIL\_FROM** = **A.ai** <no-reply@yourdomain.com>.

**IMPORTANT**

The production API will not start without **RESEND\_API\_KEY**. Do not keep the default sender **onboarding@resend.dev**: it only delivers to your own Resend account email, so real users would never get verification emails.

**Phase F: AI provider keys****Step 16 Create at least one AI provider key**

| Provider                        | Where                             | Variable                  |
|---------------------------------|-----------------------------------|---------------------------|
| Google Gemini                   | aistudio.google.com → Get API key | <b>GEMINI_API_KEY</b>     |
| Groq (also enables voice input) | console.groq.com → API Keys       | <b>GROQ_API_KEY</b>       |
| OpenRouter                      | openrouter.ai → Keys              | <b>OPENROUTER_API_KEY</b> |

Providers without a key are simply switched off. With **no** key at all, the API's **/ready** check fails. Check each provider's rate limits: free keys run out quickly when many people use the site.

**Phase G: the login secret****Step 17 Generate JWT\_SECRET**

Run this in any terminal and copy the output:

```
node -e "console.log(require('crypto').randomBytes(48).toString('base64url'))"
```

This is `JWT_SECRET` (must be at least 32 characters). Use a different value for staging and production. Changing it later signs every user out.

## Phase H: Render (API server)

### Step 18 (Only if needed) change the region

`render.yaml` uses `region: singapore` for both services. If your Supabase and Upstash are in another region, edit both `region:` lines (for example `frankfurt`, `oregon`, `ohio`, `virginia`), commit and push **before** Step 19.

### Step 19 Create the services from the Blueprint

1. Go to **dashboard.render.com**, sign in with GitHub, and add a payment method (**Account Settings** → **Billing**).
2. Click **New +** → **Blueprint**.
3. Connect GitHub if asked, and give Render access to the **A.AI** repository (you can allow only this repo).
4. Select **Ajay-Dabhi-99/A.AI**, branch **main**. Blueprint name: **a-ai**.
5. Render reads `render.yaml` and shows two services: **a-ai-api** (web service) and **a-ai-attachments-cleanup** (cron job).
6. Render now asks for every value marked `sync: false`. Fill them in using the table in Step 20.
7. Click **Apply / Deploy Blueprint**.

### Step 20 Fill in the environment variables

**a-ai-api** needs:

| Variable                                                                                     | Value                                                         |
|----------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| <code>DATABASE_URL</code>                                                                    | Production pooled string, port 6543 (Step 7)                  |
| <code>DIRECT_URL</code>                                                                      | Production direct string, port 5432 (Step 7)                  |
| <code>REDIS_URL</code>                                                                       | Production <code>rediss://</code> URL (Step 12)               |
| <code>JWT_SECRET</code>                                                                      | From Step 17                                                  |
| <code>CORS_ORIGIN</code>                                                                     | <code>https://app.yourdomain.com</code> (no slash at the end) |
| <code>APP_URL</code>                                                                         | <code>https://app.yourdomain.com</code>                       |
| <code>RESEND_API_KEY</code>                                                                  | From Step 15                                                  |
| <code>EMAIL_FROM</code>                                                                      | <b>A.ai</b> <no-reply@yourdomain.com>                         |
| <code>SUPABASE_URL</code>                                                                    | From Step 9                                                   |
| <code>SUPABASE_SERVICE_ROLE_KEY</code>                                                       | From Step 9                                                   |
| <code>GEMINI_API_KEY</code> , <code>GROQ_API_KEY</code> ,<br><code>OPENROUTER_API_KEY</code> | The keys you have (Step 16); leave the others empty           |
| <code>SENTRY_DSN</code>                                                                      | Optional (Section 5); leave empty to disable                  |

**a-ai-attachments-cleanup** needs `DATABASE_URL`, `SUPABASE_URL` and `SUPABASE_SERVICE_ROLE_KEY` (same values as above).

These are already set by `render.yaml`, do not change them: `NODE_VERSION=22.16.0`, `NODE_ENV=production`, `HOST=0.0.0.0`, `LOG_LEVEL=info`, `TRUST_PROXY_HOPS=1`, `SUPABASE_STORAGE_BUCKET=a-ai-attachments`, `SENTRY_ENVIRONMENT=production`. Render adds `PORT` and `RENDER_GIT_COMMIT` itself.

**Check the first deploy:** open **a-ai-api** → **Events / Logs**. You should see the build, then `pnpm db:deploy` applying the migrations, then the server starting. If it stops with "Invalid environment configuration", the log lists which variable is wrong (never its value); see Section 8.

Test it using the temporary Render address shown at the top of the service page:

```
curl https://a-ai-api-xxxx.onrender.com/health
curl https://a-ai-api-xxxx.onrender.com/ready
```

**Expected:** `/health` returns `"status":"ok"`; `/ready` returns HTTP 200 with status `ready`.

### Step 21 Add the API custom domain

1. **a-ai-api** → **Settings** → **Custom Domains** → **Add Custom Domain:** `api.yourdomain.com`.
2. Render shows a **CNAME** target (like `a-ai-api-xxxx.onrender.com`). On your registrar's DNS page add: Type **CNAME**, Name/Host `api`, Value = that target.
3. Back in Render click **Verify**. Wait until the certificate shows as issued.
4. Check: open `https://api.yourdomain.com/health` in the browser.

### Step 22 Copy the Render deploy hook

1. **a-ai-api** → **Settings** → **Deploy Hook**. Copy the URL.
2. Save it as `RENDER_DEPLOY_HOOK_URL`. Anyone with this URL can trigger a deploy, so treat it like a password.

#### NOTE

Render deploys the latest commit on `main` when this hook is called. `autoDeploy: false` in `render.yaml` means a normal `git push` does **not** deploy.

### Step 23 (Optional) validate the settings without showing them

Open **a-ai-api** → **Shell** and run:

```
pnpm check:env
```

It prints only whether the configuration is valid and which AI providers are set. It never prints secret values.

## Phase I: Vercel (website)

### Step 24 Import the same repository into Vercel

1. Go to **vercel.com** → **Sign up with GitHub** → **Add New...** → **Project**.
2. Give Vercel access to **A.AI** and click **Import**.
3. Project name: `a-ai-web`.
4. **Root Directory:** click **Edit** and choose `apps/web`. This is the key setting for a single repo.
5. Leave Framework, Build Command, Output Directory and Install Command as they are. `apps/web/vercel.json` already sets them (it goes back to the repo root, installs the workspace and builds only the website).

### Step 25 Add the website environment variables

Still on the import screen, open **Environment Variables** and add:

| Name                         | Value                                   | Notes                          |
|------------------------------|-----------------------------------------|--------------------------------|
| <code>VITE_API_URL</code>    | <code>https://api.yourdomain.com</code> | No slash at the end, no path   |
| <code>VITE_SENTRY_DSN</code> | Optional                                | Browser Sentry DSN (Section 5) |

Click **Deploy**. The first build takes a few minutes.

#### IMPORTANT

Values starting with **VITE\_** are built into the website and anyone can read them. Never put a secret here. If you change one later, you must redeploy for it to take effect.

### Step 26 Stop Vercel from deploying on every push

So that the website only goes live together with the API from the Deploy workflow, turn off Git-triggered deployments. Add this to `apps/web/vercel.json` (next to `"framework"`), commit and push:

```
"git": { "deploymentEnabled": false },
```

This is a documented Vercel setting. After your first Deploy workflow run (Step 34), open **Vercel** → **Deployments** and confirm a new deployment appeared from the deploy hook. If it did not, remove this line again and let Vercel deploy on push instead (then the website updates when you merge to `main`, and the API updates when you run the Deploy workflow).

### Step 27 Create the Vercel deploy hook

1. **a-ai-web** → **Settings** → **Git** → **Deploy Hooks**.
2. Name: `github-deploy`, Branch: `main`. Click **Create Hook**.
3. Copy the URL and save it as `VERCEL_DEPLOY_HOOK_URL` (secret).

### Step 28 Add the website custom domain

1. **a-ai-web** → **Settings** → **Domains** → **Add**: `app.yourdomain.com`.
2. Vercel shows a record, usually Type `CNAME`, Name `app`, Value `cname.vercel-dns.com` (use exactly what Vercel shows). Add it at your registrar.
3. Wait until Vercel shows **Valid Configuration**.
4. Check: open `https://app.yourdomain.com`. The A.ai home page loads.

## Phase J: DNS check

### Step 29 Confirm all DNS records

| Name / Host      | Type                      | Value           | Added in |
|------------------|---------------------------|-----------------|----------|
| <code>app</code> | CNAME (or as Vercel says) | Vercel target   | Step 28  |
| <code>api</code> | CNAME                     | Render target   | Step 21  |
| Resend records   | TXT / CNAME / MX          | As Resend shows | Step 14  |

If you use Cloudflare DNS, set the `app` and `api` records to **DNS only** (grey cloud) so Vercel and Render can issue certificates.

## Phase K: GitHub settings

### Step 30 Create the "test" environment

1. Open `github.com/Ajay-Dabhi-99/A.AI` → **Settings** → **Environments** → **New environment**. Name: `test`.
2. Under **Environment secrets** → **Add secret**, add: `TEST_DATABASE_URL`, `TEST_DIRECT_URL`, `TEST_REDIS_URL` (from Steps 11 and 13).

### Step 31 Create the "production" environment

1. **New environment**, name: `production`.
2. Tick **Required reviewers** and add yourself, then **Save protection rules**. (Available because the repository is public.) Every production deploy will now wait for your approval.

3. **Environment secrets:** `RENDER_DEPLOY_HOOK_URL` (Step 22) and `VERCEL_DEPLOY_HOOK_URL` (Step 27).
4. **Environment variables:** `API_URL` = `https://api.yourdomain.com` and `WEB_ORIGIN` = `https://app.yourdomain.com`.

### Step 32 Protect the main branch

1. **Settings** → **Branches** → **Add branch ruleset** (or **Add rule**), target `main`.
2. Enable **Require a pull request before merging**.
3. Enable **Require status checks to pass**, and add `Typecheck`, `lint`, `test`, `build` and `Browser smoke` (`Playwright`, `mocked API`).
4. Save.

### Step 33 Check the Deploy workflow is visible

Open **Actions**. In the left list you should see **CI**, **Deploy** and **Test**. The Deploy workflow is run only by hand.

## Phase L: first release

### Step 34 Run the Deploy workflow

1. **Actions** → **Deploy** → **Run workflow**.
2. Branch: `main`. Target: `production`. Click **Run workflow**.
3. The **Full gate** job runs first (about 10–20 minutes): `pnpm verify`, browser smoke tests and live tests against the test database.
4. The **Deploy to production** job then waits for your approval. Click **Review deployments** → `production` → **Approve and deploy**.

### Step 35 Watch it finish

The deploy job:

- checks that the hook URLs and addresses are configured (values are hidden);
- calls the Render and Vercel deploy hooks;
- waits up to 15 minutes until `https://api.yourdomain.com/health` reports this commit;
- runs the release smoke test (it calls `no AI` provider, so it costs nothing).

**Expected:** the log ends with `All N checks passed`. and the job is green. Meanwhile you can watch the build in **Render** → `a-ai-api` → **Events** and **Vercel** → **Deployments**.

### Step 36 Run the smoke test yourself (optional)

```
pnpm smoke --api https://api.yourdomain.com --web https://app.yourdomain.com
```

Each line shows `PASS` or `FAIL`: health, readiness, security headers, error format, CORS, cookie flags, model list, website CSP and deep links.

### Step 37 Test the live site by hand

1. Open `https://app.yourdomain.com` in a private window.
2. Chat as a guest; keep going until the daily guest limit message appears (20 messages by default).
3. Sign up with your first and last name; open the verification email and click the link.
4. Sign in, chat, reload the page: the conversation is still there.
5. Run a comparison of two or more models.
6. Upload an image (if Supabase Storage is set) and try the microphone (if Groq is set).
7. Open history and the dashboard.

## Phase M: after the first release

### Step 38 Make yourself admin

Sign up and verify your email first. Then open **Render** → **a-ai-api** → **Shell** and run:

```
pnpm admin:promote your-email@example.com
```

There is deliberately no button or API for this. To remove admin rights: `pnpm admin:promote your-email@example.com --revoke`.

### Step 39 Run the cleanup job once

**Render** → **a-ai-attachments-cleanup** → **Trigger Run**. Check the log finishes without errors. After that it runs daily at 03:30 UTC.

### Step 40 Model list (nothing to do)

The API fills the model registry with the default models the first time it is used. If you want to do it explicitly, run `pnpm db:seed` in the Render Shell; it never overwrites changes made in the admin screen.

### Step 41 Record the release

Add to `docs/development/roadmap.md`: the commit SHA, date, Render deploy ID, Vercel deployment URL and the smoke test output. Note the previous Render deploy and Vercel deployment: those are your rollback targets.

## 4. Releasing updates later

Once the first deploy works, every future release follows the same short routine from your single repository:

1. Create a branch: `git checkout -b feature/my-change`.
2. Make the change, then run `pnpm verify` locally.
3. Push and open a pull request into `main`. Wait for CI to go green.
4. Merge the pull request.
5. **Actions** → **Deploy** → **Run workflow** → **production**, then approve.
6. Wait for the green smoke test, check the site quickly, and record the release.

### 4.1 Database changes

If your change adds a Prisma migration, you do nothing extra: Render runs `pnpm db:deploy` before the new version receives traffic. Follow these rules so that a rollback stays safe:

- Only add: new tables, new nullable columns, columns with defaults, new indexes.
- Remove or rename a column only in a later release, after no live version uses it.
- Never edit a migration that has already been deployed. Write a new one instead.
- Every new table must enable Row Level Security in the same migration.
- Run `pnpm test:live` locally when database or Redis behaviour changes.

### 4.2 Changing a secret

- API secret: **Render** → **a-ai-api** → **Environment** → **edit** → **Save**. Render restarts the service.
- Website value: **Vercel** → **Settings** → **Environment Variables**, then redeploy (a `VITE_` value only changes after a new build).
- Deploy hooks / test URLs: **GitHub** → **Settings** → **Environments**.

## 5. Optional: error monitoring with Sentry

1. Create an account at **sentry.io** and two projects: platform **Node.js** (name **a-ai-api**) and platform **React** (name **a-ai-web**).
2. Copy each project's **DSN** (Project Settings → Client Keys).
3. Render → a-ai-api → Environment: **SENTRY\_DSN** = Node DSN.
4. Vercel → Environment Variables: **VITE\_SENTRY\_DSN** = React DSN, then redeploy.
5. In both Sentry projects keep **Data Scrubber** and **Scrub IP Addresses** switched on.

Without a DSN nothing is sent. With a DSN only unexpected errors are reported, tagged with the **requestId**; headers, cookies, request bodies and query strings are removed first.



## 6. Optional: adding a staging site from the same repo

Staging is a second, private copy of the whole system for testing a release before production. It uses the **same repository and the same `main` branch**; only the services and secrets differ. The Deploy workflow already supports it (target `staging`).

1. Create separate resources: a Supabase project `a-ai-staging` (with its own `a-ai-attachments` bucket) and an Upstash database `a-ai-staging`. (A free Supabase account allows a limited number of projects; staging may need a paid plan or a second organization.)
2. Render: **New +** → **Web Service** (not Blueprint, because the Blueprint names are already used) from the same repo and branch. Name `a-ai-api-staging`; copy the Build Command, Pre-Deploy Command, Start Command and Health Check Path (`/health`) from `render.yaml`; turn **Auto-Deploy off**; add the same environment variables with the staging values and `NODE_ENV=production`, `TRUST_PROXY_HOPS=1`, `NODE_VERSION=22.16.0`, `SENTRY_ENVIRONMENT=staging`.
3. Render: custom domain `api.staging.yourdomain.com`; copy its deploy hook.
4. Vercel: **Add New** → **Project**, import the **same** repo again, name `a-ai-web-staging`, Root Directory `apps/web`, `VITE_API_URL=https://api.staging.yourdomain.com`, domain `app.staging.yourdomain.com`, and a deploy hook for `main`.
5. GitHub: create environment `staging` with secrets `RENDER_DEPLOY_HOOK_URL`, `VERCEL_DEPLOY_HOOK_URL` (the staging hooks) and variables `API_URL`, `WEB_ORIGIN` (the staging addresses).
6. Release routine becomes: **Deploy** → **staging**, test by hand, then **Deploy** → **production**.

### TIP

Because `app.staging.yourdomain.com` and `api.staging.yourdomain.com` share the same parent domain, sign-in cookies work on staging too.

## 7. Rollback, backups and restore

### 7.1 Rolling back a bad release

1. API: **Render** → **a-ai-api** → **Events**, find the previous successful deploy, click **Rollback**. Migrations are not run again.
2. Website: **Vercel** → **a-ai-web** → **Deployments**, open the previous production deployment, click **Instant Rollback**.
3. Run `pnpm smoke --api https://api.yourdomain.com --web https://app.yourdomain.com. /health` shows which commit is live.
4. Open a GitHub issue with the request IDs (from the `x-request-id` header) or Sentry events that showed the problem.

Because migrations only add things, rolling the code back never requires rolling the database back.

### 7.2 Backups

| Data                                  | Backup                                                          | How to restore                                                                                 |
|---------------------------------------|-----------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| PostgreSQL (accounts, chats, history) | Supabase daily backups (plus Point-in-Time Recovery if enabled) | Supabase → Database → Backups → Restore. Pause releases, restore, run the smoke test.          |
| Redis (sessions, quotas, rate limits) | None needed (short-lived)                                       | Nothing to restore; guests just get new sessions. Signed-in sessions are stored in PostgreSQL. |
| Uploaded images and videos            | Not included in database backups                                | Lost files show as "Unavailable". Add a scheduled bucket copy if they matter.                  |

Practise a restore on staging (or the test project) before launch and every three months.

## 8. Troubleshooting

The API checks every setting when it starts. If something is wrong it stops and prints the variable names (never the values) in the Render log. Always look there first.

| What you see                                                               | What to do                                                                                                                                         |
|----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| REDIS_URL: production requires TLS (rediss://)                             | Use the Upstash URL that starts with <code>rediss://</code> .                                                                                      |
| CORS_ORIGIN.0: production origins must use https://                        | Set exactly <code>https://app.yourdomain.com</code> , no slash or path at the end.                                                                 |
| RESEND_API_KEY: production requires a real email provider                  | Add the Resend key (Step 15).                                                                                                                      |
| JWT_SECRET: must be at least 32 characters                                 | Generate a new one (Step 17).                                                                                                                      |
| set SUPABASE_URL and SUPABASE_SERVICE_ROLE_KEY together                    | Fill both, or clear both (uploads then switch off).                                                                                                |
| DATABASE_URL: must be a postgres:// ... connection string                  | Copy the string again; check nothing was cut off and the password is filled in.                                                                    |
| Pre-deploy step <code>pnpm db:deploy</code> fails                          | Check <code>DIRECT_URL</code> (port 5432, correct password, special characters encoded).                                                           |
| Build fails: <code>tsc</code> or <code>turbo</code> not found              | The build command must keep <code>NODE_ENV=development</code> <code>pnpm install</code> , as in <code>render.yaml</code> .                         |
| <code>/health</code> OK but <code>/ready</code> returns 503                | The response names the failing part: database, Redis, or no AI provider key.                                                                       |
| Sign-in seems to work but you are signed out immediately; guest chat fails | Website and API are not on the same parent domain (Section 1.2), or you are using the <code>.vercel.app</code> address.                            |
| Browser console shows CORS errors                                          | <code>CORS_ORIGIN</code> on Render must exactly match the website address, including <code>https://</code> and <code>app..</code>                  |
| Website calls the wrong API / "Network error"                              | Fix <code>VITE_API_URL</code> in Vercel, then redeploy.                                                                                            |
| Verification emails never arrive                                           | Domain not verified in Resend, or <code>EMAIL_FROM</code> still uses <code>onboarding@resend.dev</code> . Check the spam folder and Resend → Logs. |
| Deploy workflow: "is not configured for the production environment"        | A hook secret or <code>API_URL</code> / <code>WEB_ORIGIN</code> variable is missing in GitHub → Environments → production.                         |
| Deploy workflow: live tests fail with "need TEST_DATABASE_URL"             | Add the <code>test</code> environment secrets (Step 30).                                                                                           |
| Deploy workflow: "The new build did not come up in time"                   | The Render build or migration failed; open Render → a-ai-api → Events.                                                                             |
| Image upload button missing                                                | Supabase Storage variables not set, or the bucket <code>a-ai-attachments</code> does not exist.                                                    |
| Microphone button missing                                                  | <code>GROQ_API_KEY</code> is not set.                                                                                                              |
| Everyone hits rate limits at the same time                                 | <code>TRUST_PROXY_HOPS</code> must be <code>1</code> on Render.                                                                                    |
| Custom domain certificate never issues                                     | Check the CNAME value; with Cloudflare, switch the record to DNS only.                                                                             |

Every error the API returns contains a `requestId` (also in the `x-request-id` response header). Search for it in **Render** → **Logs**, or in Sentry by the `requestId` tag.

## Appendix A: all API environment variables

| Variable                                            | Required         | Default / example                                      |
|-----------------------------------------------------|------------------|--------------------------------------------------------|
| NODE_ENV                                            | yes              | production (render.yaml)                               |
| PORT, HOST                                          | auto             | Render sets PORT; HOST=0.0.0.0                         |
| LOG_LEVEL                                           | no               | info                                                   |
| DATABASE_URL                                        | yes              | Supabase pooled, port 6543                             |
| DIRECT_URL                                          | yes              | Supabase direct, port 5432                             |
| REDIS_URL                                           | yes              | rediss://...                                           |
| JWT_SECRET                                          | yes              | 32+ characters                                         |
| CORS_ORIGIN                                         | yes              | https://app.yourdomain.com<br>(comma-separate several) |
| APP_URL                                             | yes              | https://app.yourdomain.com                             |
| RESEND_API_KEY                                      | yes (production) | Resend key                                             |
| EMAIL_FROM                                          | yes              | A.ai <no-reply@yourdomain.com>                         |
| GEMINI_API_KEY, GROQ_API_KEY,<br>OPENROUTER_API_KEY | at least one     | Provider keys                                          |
| SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY             | for uploads      | Both or neither                                        |
| SUPABASE_STORAGE_BUCKET                             | no               | a-ai-attachments                                       |
| TRUST_PROXY_HOPS                                    | no               | 1 on Render                                            |
| SENTRY_DSN, SENTRY_ENVIRONMENT                      | no               | Empty = off                                            |
| GUEST_SESSION_TTL_MINUTES                           | no               | 1440 (5–10080)                                         |
| GUEST_DAILY_MESSAGE_LIMIT                           | no               | 20 (1–1000)                                            |
| USER_DAILY_MESSAGE_LIMIT                            | no               | 200 (1–100000)                                         |
| GUEST_COMPARE_MAX_MODELS                            | no               | 2 (2–4)                                                |
| USER_COMPARE_MAX_MODELS                             | no               | 4 (2–4)                                                |
| CONTEXT_SUMMARY_ENABLED                             | no               | true                                                   |
| CHAT_FALLBACK_ENABLED                               | no               | true                                                   |
| ATTACHMENT_MAX_BYTES                                | no               | 5242880 (5 MiB)                                        |
| AUDIO_MAX_BYTES                                     | no               | 10485760 (10 MiB)                                      |
| VIDEO_MAX_BYTES                                     | no               | 52428800 (50 MiB)                                      |
| RENDER_GIT_COMMIT                                   | auto             | Set by Render; shown by /health                        |

Website (Vercel): VITE\_API\_URL (required), VITE\_SENTRY\_DSN (optional).

## Appendix B: useful commands

| Command                                                     | What it does                                             |
|-------------------------------------------------------------|----------------------------------------------------------|
| <code>pnpm install</code>                                   | Install everything (also generates the Prisma client)    |
| <code>pnpm verify</code>                                    | Format, lint, typecheck, unit + integration tests, build |
| <code>pnpm test:e2e</code>                                  | Browser smoke tests (needs Chromium)                     |
| <code>pnpm test:live</code>                                 | Tests against the real test database and Redis           |
| <code>pnpm check:env</code>                                 | Validate environment settings without printing values    |
| <code>pnpm db:deploy</code>                                 | Apply migrations (Render runs this automatically)        |
| <code>pnpm db:seed</code>                                   | Add default models to the registry (optional)            |
| <code>pnpm admin:promote &lt;email&gt; [--revoke]</code>    | Grant or remove admin rights                             |
| <code>pnpm attachments:cleanup</code>                       | Delete expired uploads (the daily cron job)              |
| <code>pnpm smoke --api &lt;url&gt; --web &lt;url&gt;</code> | Check a deployed release; calls no AI provider           |

## Appendix C: final checklist

- [ ] Domain bought; **app.** and **api.** records point to Vercel and Render
- [ ] Supabase production project, private bucket **a-ai-attachments**, backups checked
- [ ] Supabase test project
- [ ] Upstash production and test databases (**rediss://**)
- [ ] Resend domain verified, API key created, **EMAIL\_FROM** on your domain
- [ ] At least one AI provider key; limits checked
- [ ] Render Blueprint applied, all secrets filled, **/ready** returns ready, deploy hook copied
- [ ] Vercel project with Root Directory **apps/web**, **VITE\_API\_URL** set, domain valid, deploy hook copied
- [ ] GitHub environments **test** and **production** filled; **main** protected
- [ ] Deploy workflow green, smoke test passed, manual checks done
- [ ] Admin promoted, cleanup job run once, release recorded in roadmap.md